

Failure (a real rollback trigger) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Every example below plays back this same simulated score sequence — a healthy start, then a sustained drop partway through, acting out the exact situation this exercise asks for:

```
scores = [0.95, 0.94, 0.96, 0.93, 0.95, 0.60, 0.55, 0.58, 0.52, 0.50]
baseline = 0.95
```

Basic Version

Approach 1 — the direct way

```
def should_rollback(scores_so_far, baseline, window_size=5, drop_threshold=0.2):
    if len(scores_so_far) < window_size:
        return False
    recent = scores_so_far[-window_size:]
    avg_recent = sum(recent) / len(recent)
    percent_drop = (baseline - avg_recent) / baseline
    return percent_drop >= drop_threshold

def rollback_to_previous():
    print("ROLLBACK: reverting to last known-good version")
    return "v1"

scores = [0.95, 0.94, 0.96, 0.93, 0.95, 0.60, 0.55, 0.58, 0.52, 0.50]
seen = []
for i, s in enumerate(scores):
    seen.append(s)
    if should_rollback(seen, baseline=0.95):
        print(f"request {i}: score {s} -- triggering rollback")
        rollback_to_previous()
```

Expected output:

```
request 7: score 0.58 -- triggering rollback
ROLLBACK: reverting to last known-good version
```

```
request 8: score 0.52 -- triggering rollback  
ROLLBACK: reverting to last known-good version  
request 9: score 0.5 -- triggering rollback  
ROLLBACK: reverting to last known-good version
```

It correctly waits until request 7 (the 8th score) before firing — that's the first point where the 5 most recent scores ([0.93, 0.95, 0.60, 0.55, 0.58] , averaging 0.722) are far enough below the 0.95 baseline (a 24% drop) to clear the 20% threshold. This version works correctly for what the exercise asks, but it calls `rollback_to_previous()` three separate times for what is really one decline — that's the flapping problem, fixed below.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — type hints, named parameters, same repeat-firing behavior

```
def should_rollback(
    scores_so_far: list[float],
    baseline: float,
    window_size: int = 5,
    drop_threshold: float = 0.2,
) -> bool:
    if len(scores_so_far) < window_size:
        return False
    recent = scores_so_far[-window_size:]
    avg_recent = sum(recent) / len(recent)
    percent_drop = (baseline - avg_recent) / baseline
    return percent_drop >= drop_threshold

def rollback_to_previous() -> str:
    print("ROLLBACK: reverting to last known-good version")
    return "v1"

def watch_and_rollback(scores: list[float], baseline: float) -> None:
    seen: list[float] = []
    for i, s in enumerate(scores):
        seen.append(s)
        if should_rollback(seen, baseline):
            print(f"request {i}: score {s} -- triggering rollback")
            rollback_to_previous()

scores = [0.95, 0.94, 0.96, 0.93, 0.95, 0.60, 0.55, 0.58, 0.52, 0.50]
watch_and_rollback(scores, baseline=0.95)
```

Expected output: identical to Basic's — fires at request 7, and again at 8 and 9, since `should_rollback` still has no memory between calls.

Difference from Basic: Approach 1 adds full type hints and separates the watching loop into its own `watch_and_rollback` function, but the core check is unchanged, so it still flaps exactly the way Basic does. That's intentional — it shows the flapping problem is about *state*, not about writing the `if` more carefully, which is why Advanced fixes it with a class instead of a better function.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a `RollbackMonitor` that fires once per decline

```
class RollbackMonitor:
    def __init__(self, baseline: float, window_size: int = 5, drop_threshold: flo
        self.baseline = baseline
        self.window_size = window_size
        self.drop_threshold = drop_threshold
        self.scores: list[float] = []
        self.already_rolled_back = False

    def record_score(self, score: float) -> bool:
        self.scores.append(score)
        if self.already_rolled_back:
            return False
        if len(self.scores) < self.window_size:
            return False

        recent = self.scores[-self.window_size:]
        avg_recent = sum(recent) / len(recent)
        percent_drop = (self.baseline - avg_recent) / self.baseline

        if percent_drop >= self.drop_threshold:
            self.already_rolled_back = True
            return True
        return False

    def rollback_to_previous() -> str:
        print("ROLLBACK: reverting to last known-good version")
        return "v1"

scores = [0.95, 0.94, 0.96, 0.93, 0.95, 0.60, 0.55, 0.58, 0.52, 0.50]
monitor = RollbackMonitor(baseline=0.95)

for i, s in enumerate(scores):
    if monitor.record_score(s):
        print(f"request {i}: score {s} -- triggering rollback")
        rollback_to_previous()
```

Expected output:

```
request 7: score 0.58 -- triggering rollback
ROLLBACK: reverting to last known-good version
```

Exactly one rollback, at the same request (7) both earlier approaches first caught — but this time nothing fires again at request 8 or 9, because `self.already_rolled_back` is now `True` and every later call to `record_score` short-circuits before re-checking the window.

Approach 2 — resetting the monitor after a fresh deploy

```
def redeploy(monitor: RollbackMonitor, new_baseline: float) -> RollbackMonitor:
    """A new version was deployed after the rollback – start watching fresh."""
    return RollbackMonitor(
        baseline=new_baseline,
        window_size=monitor.window_size,
        drop_threshold=monitor.drop_threshold,
    )

monitor = RollbackMonitor(baseline=0.95)
scores_before = [0.95, 0.94, 0.96, 0.93, 0.95, 0.60, 0.55, 0.58]
for i, s in enumerate(scores_before):
    if monitor.record_score(s):
        print(f"request {i}: rollback fired, already_rolled_back = {monitor.already_rolled_back}")

# a fixed version (v3) gets deployed after the rollback – fresh monitor, fresh baseline
monitor = redeploy(monitor, new_baseline=0.93)
print("new monitor already_rolled_back:", monitor.already_rolled_back)
print("new monitor scores:", monitor.scores)
```

Expected output:

```
request 7: rollback fired, already_rolled_back = True
new monitor already_rolled_back: False
new monitor scores: []
```

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate's `should_rollback` is a pure function with no memory — correct for a single check, but calling it in a loop re-triggers on every request once the condition stays true. Approach 1 fixes that by moving the score history and a fired flag *into* a `RollbackMonitor` object, so the object itself remembers what it already did. Approach 2 completes the picture: a fired monitor should not stay silent forever — once a new version is actually deployed (a genuine "we're trying again"), a fresh `RollbackMonitor`

should replace the old one, with its own clean baseline and an empty score history, ready to catch the *next* decline if there is one.

Which one should you actually write? Approach 1's `RollbackMonitor` is the right shape for the Build Task's `rollback.py` — a bare function like Basic/Intermediate's `should_rollback` is fine for a one-off script, but the moment this runs continuously against real traffic (which is the entire point of a rollback trigger), it needs to remember it already acted, exactly like Approach 1 does. Add Approach 2's reset-on-redeploy the moment you wire this up to a real `deploy()` call, so a rollback trigger doesn't stay permanently tripped after the problem's already been fixed.